WebSDK 2.0

Part 1

The Basics

Introduction

If you have developed custom controls for the original Xojo Web Framework (originally released in Fall 2010) then you know that Web 2.0 represents a major upgrade in the capabilities and functionality of the Xojo Web Framework. That said, a lot has changed in the last 10 years, both within Xojo and in internet technologies and browser capabilities at large.

One thing that was done differently this time is that having an SDK available for developers to add capabilities to our framework was planned from day one which gave us an opportunity to fix a number of things that made it downright impossible to achieve certain things before. With this flexibility does come a bit of complexity, but the good news is that this time our controls are built nearly identically to the way you will build your controls going forward.

Section 1

Basic Concepts

Building custom controls for use with the Xojo Web framework requires two separate code bases working in concert with one another... one in Xojo and one in JavaScript. When an end-user connects to a Xojo Web app using their browser, your custom control's Xojo code is initialized when an instance of your control is created for that user's session. Each control is given a single opportunity per session to send its javascript framework code and its css information to the browser. Any per-control customization must be done either by your javascript class or through the use of the Xojo command ExecuteJavascript.

We have instilled a separation of logic in the Web 2.0 framework such that no HTML or DOM rendering is done on the server side once the frameworks have been initialized on the browser. That is, the individual javascript classes are responsible for all of the drawing of their respective controls. This is a change from the way things were done in Web 1.0 for two major reasons. First, we found that having some of the rendering code on the server and some in the JavaScript code made it very hard to debug. Secondly it reduces the amount of traffic that is sent from the server to the browsers at runtime by a *significant* amount. We urge you to follow this same model when designing your Web 2.0 controls to give your users and their users the best experience possible.

Another change that was made was in how the Xojo JavaScript frameworks are assembled before delivery to the browser. In the past, the framework only included the code for the controls you had used on a layout or in code. While it meant that the framework delivery was as small as it could be, it also meant that the first time you added a control to your project that you hadn't used before, all of the end-users would be required to download a new version of the framework the next time they connected. In Web 2.0 the entire Xojo JavaScript framework has been minified so that the first time one of your users connects to the app, they get the whole framework for every control that is included in the Xojo framework and the only time a user will need to re-download the Xojo portion of the frameworks is if *we* make a change or add a built-in control.

Namespaces

For the purposes of avoiding conflicts between framework code, your code and anything else that you might put in your application, you must use a namespace for your controls.

Please refer to the Namespaces section for more information.

Control Naming Conventions

To prevent name conflicts and to make it easier to identify in lists, you should prefix your classes. For example, a company named Acme might name their controls Acme_Button or Acme_SourceList.

Event Order

Understanding the initial sequence of events that fire when your control is created and then added to a web page is crucial for successful deployment and the Xojo developer experience.

Controls

There are two different kinds of fully custom controls which can be created for Xojo Web 2.0... Visual Controls (like WebButton) and Non-Visual Controls (like WebTimer). Your Xojo and Javascript code will both be implemented as Classes which extend a built-in class (see Figure 1)

Control Supers		
	Non-Visual	Visual
Xojo Code	WebSDKControl	WebSDKUIControl
Javascript Code	XojoControl	XojoVisualControl

Figure 1

For the most part, you will be creating visual controls. That is, controls which have a presence in the DOM and present a visual interface to the end-user. Only controls which do not show any UI to the end-user should be built as non-visual controls.

Section 2

Your First Control

For your control to work in the Xojo Web control ecosystem, there are several methods and events that you *must* implement in your controls.

JavaScript

The JavaScript portion of Xojo Web Framework 2.0 is written entirely in EcmaScript 6. This adds some protections for code at runtime, but also imposes some restrictions on how your code must be formatted for success.

When redesigning the JavaScript framework, we decided to use Microsoft® Visual Studio Code and TypeScript as our programming environment and while you are not required to use it in your workflow, it does offer some code completion and type checking that you would otherwise not have.

First, you will need to define a namespace for yourself. Xojo maintains a list of namespaces in an attempt to avoid conflicts with other developers. If you will be writing controls which you will be selling or providing to other developers, please contact us to check to make sure your namespace is available. Note that namespaces which begin with Xojo (upper or lowercase) are reserved for our use. See the section on Namespaces for more information.

For this exercise we will be creating a generic Bootstrap button named “MyButton” in a namespace called “Example”.

Your control should be a subclass of the XojoVisualControl class which is in the XojoWeb namespace. The declaration of the code in TypeScript will look like this (a pure JavaScript version is available at the end of this section).

```
namespace Example {
  export class MyButton extends XojoWeb.XojoVisual-
Control {

  }
}
```

Once you have the namespace and class, you will need to add the methods necessary for your control to work in our ecosystem.

The **constructor** method is **required**. The Xojo framework will generate a controlID for you and pass it and the events that were implemented at design-time when the control is first initialized. You *must* call super(id, events) to register your control.

```
constructor(id: string, events: string[]) {
    super(id, events);
}
```

The **updateControl** method is also **required**. It is the mechanism by which the values encoded in the Serialize event are sent to your control instances on the browser, such as position and size information, locking, indicator color, etc. The data parameter contains a JSON object containing the

current state of the control. You *must* call super.updateControl(data) for base control properties to be applied and even if you plan to override any of those properties, and you should call the super method first. Once you have updated your control properties, you should call the refresh() method to tell the framework that your control is ready for a refresh, which will be triggered asynchronously so that we may coalesce repeated calls.

```
public updateControl(data: string) {
  super.updateControl(data);
  let js = $.parseJSON(data);
  this.mCaption = js.caption;
  this.refresh();
}
```

The **render** method is **required**. This is where you should put your primary DOM rendering code. This method is called whenever a control needs to be redrawn for some reason, whether a property has changed or the view it's on has been refreshed.

For information about other methods available to you, see Part 3 - Browser Side Code.

```
public render() {
  // Have the superclass do its work
  super.render();

  // Try to get the DOM element for this control
  let el: HTMLElement | null = this.DOMElement();

  // if it doesn't exist, just return because that
  // means the control isn't fully setup yet.
  // render will be called again when it is.
  if (!el) return;

  // Sets position and user-defined locking
  this.setAttributes();

  // YOUR CODE GOES HERE

  // Apply the user's tooltip if your control
  // supports them.
  this.applyTooltip(el);

  // Lastly, apply user style changes
  this.applyUserStyle(el?);
}
```

JavaScript Example

```
var Example;
(function (Example) {
  class MyButton extends XojoWeb.XojoVisualControl {
    constructor(id, events) {
      super(id, events);
      this.mCaption = "Untitled";
    }

    render() {
      super.render();
      let el = this.DOMElement();
      if (!el)
        return;
      this.setAttributes();

      // YOUR CODE HERE

      this.replaceEveryChild(buttonWrapper);
      this.applyUserStyle();
    }

    updateControl(data) {
      super.updateControl(data);
      let js = $.parseJSON(data);
      this.mCaption = js.caption;
      this.refresh();
    }
  }
  Example.MyButton = MyButton;
})(Example || (Example = {}));
```

Section 3

Namespaces

Basic Concepts

User controls and code should not be created in the global namespace to prevent pollution of the Global JavaScript namespace. This helps reduce the possibility of conflicts with other JavaScript frameworks like jQuery and Bootstrap. To facilitate this you should have your own top-level namespace to contain your controls. Please keep in mind that namespaces which begin with "xojo" in any combination of upper or lowercase characters are reserved for our use exclusively and may cause issues for you or the users of your controls. We also reserve the namespace "example" for use in our example projects and documentation.

While the IDE currently does not check for namespace conflicts, it may in the future, so it is extremely important that you follow our guidelines in this area so that your users will not be affected when this change is made.

For the sake of clarity, namespaces must follow these conventions:

- They should be at least 3 characters in length
- May only contain letters numbers and _
- May not be a JavaScript or ECMAScript keyword.

Registering Your Namespace

Xojo is maintaining a Javascript namespace registry to help developers avoid namespace conflicts. Using the examples above, the registered namespace would be “Acme”. From then on, you would be permitted to create:

- Acme.Button
- Acme.Listbox
- Acme.SourceList

This does NOT permit you to use AcmeControls or AcmeProducts without creating a separate namespace registration.

To request a root namespace registration, send an email to WebNamespace@xojo.com and tell us what root namespace you would like to use. You will receive an email from us confirming your namespace choice.

Namespaces will be registered on a first come first served basis. We reserve the right to deny a namespace request that infringes on the trademarks of another company or would be otherwise confusing or offensive.

We will not be enforcing namespace case-sensitivity. If you register “Acme”, you may also use “acme” or “ACME” or any combination you can think of as long as the characters themselves do not change. That being said, remember that Javascript **is** case-sensitive, so you may want to just choose one and stick with it.

Part 2

Server Side Code

Section 4

WebSDKControl

WebSDKControl is a direct subclass of WebControl.

In addition to the events that are available on all subclasses of WebControl, WebSDK controls also have the following events:

Constants

APIVersion as Integer

WebSDK API version. 2020r1 = 7

Runtime Events

ExecuteEvent(name as String, parameters as JSONItem) As Boolean

This event fires when an event is sent from the browser to the app using the triggerServerEvent JavaScript method. Return True if your control has handled the event.

HandleRequest(Request as WebRequest, Response as WebResponse) As Boolean

This event fires in the same way as App.HandleURL does except that it is for requests that are *only for the current control*. To trigger this event you will need to create a specially formed URL:

```
Var url As String = "https://www.yourdomain.com/sdk/" + Self.ControlID
```

See the documentation for WebApplication.HandleURL for usage information.

JavaScriptClassName() As String

This event fires when the control is first created to send the JavaScript constructor for your control to the browser.

Serialize(js as JSONItem)

This event fires when the framework sends the first configuration to your control in the browser, whenever a property on WebControl is changed and whenever you call the Xojo UpdateBrowser method.

Session Setup Events

Events in this section fire when a session is first setting up. It is important to remember that the instance of the control that is created for this purpose is not on any page and will be destroyed immediately after the Session.PrepareSession event finishes. If you need information to persist, you should use a Shared Property.

SessionHead(session as WebSession) As String

This event fires each time a new session starts. You should return a String containing the items you wish to add to the <head> tag.

SessionJavaScriptURLs(session as WebSession) As String

This event fires each time a new session starts. You should return an array of URLs which point to any javascript you need for your control to function, whether it be a subclass of XojoControl or an external library. The Xojo framework will attempt to prevent duplicate URLs from being sent to the same session.

Hint: One reason we supply the session object is so you can figure out whether or not items have already been delivered to a particular session.

Methods

UpdateBrowser()

Similar to the method on WebControl, this method also sets the internal flag which indicates to the framework that your control needs a refresh on the browser and then immediately sends the update to the browser.

UpdateControl(sendImmediately as Boolean = False)

Similar to the UpdateBrowser method, this method also sets the internal flag which indicates to the framework that your control needs a refresh on the browser. When SendImmediately is False, any changes made are deferred until the event loop completes. When SendImmediately is True, the changes will be sent at that very moment. Note: Calling this method with SendImmediately set to True will override the "deferred" option of any changes before it was called.

Shared Methods

WebSDKControl.BrowserCompatibility(currentSession as WebSession, key as String) as Variant

Calling this method gives you access to the data that was returned by Modernizr on the user's browser. As an example, if you wanted to know if the browser supports XHR 2:

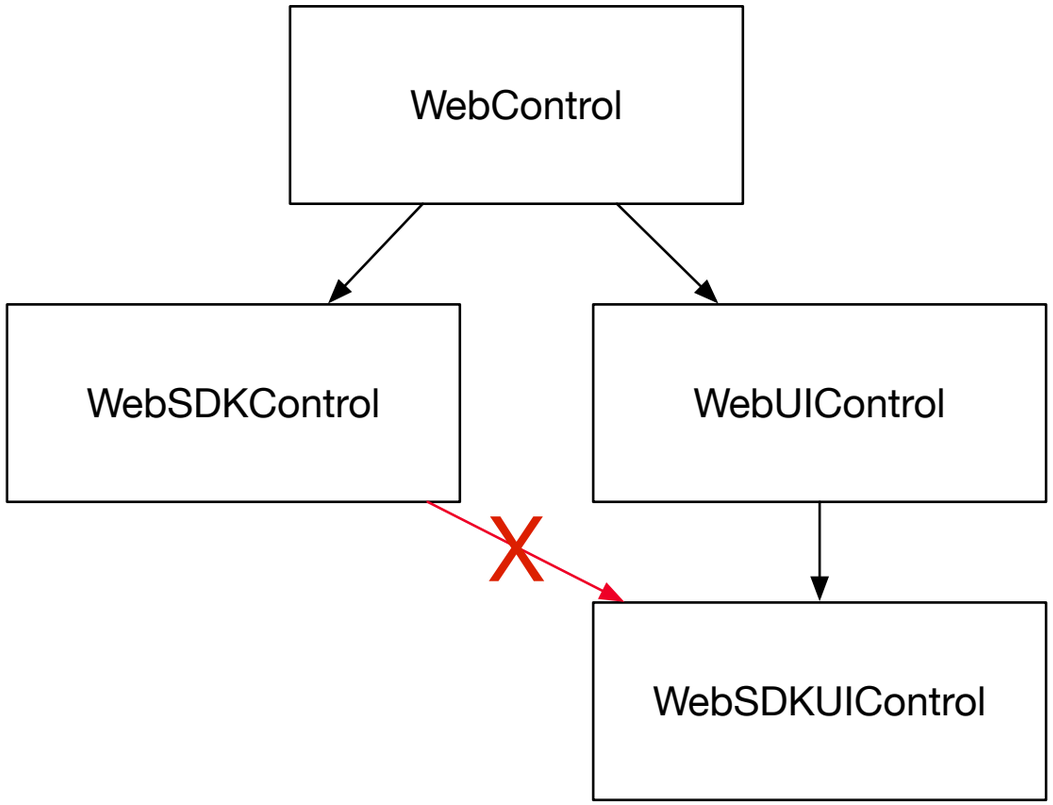
```
Var value As Boolean = WebSDKControl.BrowserCompatibility(Session, "xhr2")
```

To see a list of available keys, run a web project and open the browser's developer tools. Go to the Console and type "Modernizr" and press return. You should get a list of all of the items that we check for (which is most of them except for a few which caused problems with several of our supported browsers).

Section 5

WebSDKUIControl

WebSDKUIControl is a direct subclass of WebUIControl, but because WebUIControl inherits from WebControl it means that WebSDKUIControl doesn't actually have a direct relationship with WebSDKControl.



WebControl Inheritance Diagram

In addition to the events that are available on all subclasses of WebControl, WebSDK controls also have the following events:

Constants

NavigatorIcon

This constant is used to override the icon of your control in the Navigator. Using the WebSDKIconConverter, place the Base64 icon data into the value of this constant. – Fixed in Xojo 2020r2.

LibraryIcon

This constant is used to override the icon of your control in the Library. Using the WebSDKIconConverter, place the Base64 icon data into the value of this constant. – Fixed in Xojo 2020r2.

Runtime Events

ExecuteEvent(name as String, parameters as JSONItem) As Boolean

This event fires when an event is sent from the browser to the app using the triggerServerEvent JavaScript method. Return True if your control has handled the event.

HandleRequest(Request as WebRequest, Response as WebResponse) As Boolean

This event fires in the same way as App.HandleURL does except that it is for requests that are *only for the current control*. To trigger this event you will need to create a specially formed URL:

```
Var url As String = "https://www.yourdomain.com/sdk/" + Self.ControlID
```

See the documentation for WebApplication.HandleURL for usage.

JavaScriptClassName() As String

This event fires when the control is first created to send the JavaScript constructor for your control to the browser.

Serialize(js as JSONItem)

This event fires when the framework sends the first configuration to your control in the browser, whenever a property on WebControl is changed and whenever you call the Xojo UpdateBrowser method.

Session Setup Events

Events in this section fire when a session is first setting up. It is important to remember that the instance of the control that is created for this purpose is not on any page and will be destroyed immediately after the Session.PrepareSession event finishes. If you need information to persist, you should use a Shared Property.

SessionHead(session as WebSession) As String

This event fires each time a new session starts. You should return a String containing the items you wish to add to the <head> tag.

SessionJavaScriptURLs(session as WebSession) As String()

This event fires each time a new session starts. You should return an array of URLs which point to any javascript you need for your control to function, whether it be a subclass of XojoControl or an external library. The Xojo framework will attempt to prevent duplicate URLs from being sent to the same session.

Hint: One reason we supply the session object is so you can figure out whether or not items have already been delivered to a particular session.

SessionCSSURLs(session as WebSession) as String()

This event fires each time a new session starts. You should return an array of URLs which point to any css stylesheets you need for your control to function. The Xojo framework will attempt to prevent duplicate URLs from being sent to the same session.

Hint: One reason we supply the session object is so you can figure out whether or not items have already been delivered to a particular session.

Design Time Events

DrawControlInLayoutEditor(g as graphics)

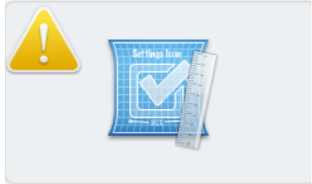
This event fires whenever your control needs to be refreshed in the IDE's layout editor. For more information of how this works, see Drawing Your Control.

Section 6

Drawing Your Control

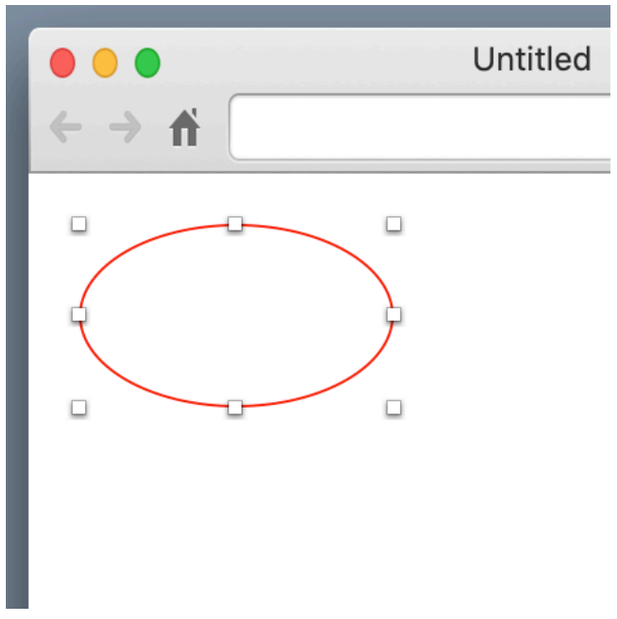
WebSDK Controls in 2020r1 and above have the ability to draw themselves in the layout editor using the same techniques that you might use to draw a control in the Paint event of a canvas. We developed a specialized XojoScript API 2 compatible Graphics context which allows you to draw just as you would in the Paint event of a canvas object. The difference here is that you also have access to the Properties and Constants of your class as well as being able to query the current Bootstrap theme.

For the purposes of drawing controls in the Layout Editor, you will need to implement the DrawControlInLayoutEditor event (you should implement it even if you're not using it, just so your end-users don't see the event). You can put just about any API 2 graphics drawing code into this event and have it render in the Layout editor. If the code you have placed in the event can't compile or runs into a runtime issue, an amber warning icon will be drawn on your control and the errors will be sent to the Messages panel.



Here's an example of how you would draw a red oval within the bounds of the control:

```
g.DrawingColor = &cFF0000
g.DrawOval 0, 0, g.width, g.height
```



In addition to the standard drawing controls, there are also some methods for accessing the properties and constants in your control as well as getting values from the user's current Bootstrap theme file.

Property Methods

BooleanProperty(Name as String) as Boolean

Gets the current value of a Boolean property of your control.

ColorProperty(Name as String) as Color

Gets the current value of a Color property of your control.

DoubleProperty(Name as String) as Double

Gets the current value of a Double property of your control.

IntegerProperty(Name as String) as Integer

Gets the current value of an Integer property of your control.

PictureProperty(Name as String) as Picture

Gets the current value of a Picture property of your control.

StringProperty(Name as String) as String

Gets the current value of a String property of your control.

Constant Methods

ConstantValue(Name as String) as String

Gets the value of a constant in your control as a String.

PictureConstant(Name as String) as Picture

Gets the value of a constant in your control as a Picture.

Color Methods

ColorGroup.NamedColor(name as String) as ColorGroup

When running on a platform that supports named colors (only macOS at the time of this writing), this method returns the named OS. Note: Color names are case sensitive, so the macOS secondary label color should be "secondaryLabelColor".

TextColor() as Color

Returns the current text color of the platform that the IDE is running on.

FillColor() as Color

Returns the current fill color of the platform that the IDE is running on.

IsDarkMode() as Boolean

Returns True if the system is running in dark mode.

Font Methods

FontCount() as Integer

Returns the number of fonts available on the system.

FontAt(index as Integer) as String

Returns the name of the font at the specified index.

CSS Methods

CSSColorValue(selector as String, propertyName as String, fallbackColor as Color) as Color

Returns a Xojo color converted from whatever format was used in the CSS color definition. Automatically converts #RRGGBB, rgb(1,0,0) and rgba(1,0,0,0.5) style colors.

CSSPixelValue(selector as String, propertyName as String, fallbackValue as Double) as Double

Returns a pixel value for a particular selector and property, making automatic adjustments for things that may be in units other than px. Useful for things that may be represented in millimeters, points or rems (rems is multiplied by the value of Body font-size).

CSSStringValue(selector as String, propertyName as String) as String

Returns the raw value of a particular selector and property.

Other Methods

DebugLog(msg as String)

Sends msg to the Messages pane. (new in 2020r2)

Part 3

Browser Side Code

Section 7

XojoControl

Before we get too far into this, it is important to note that the items listed here are what we consider the Public API. Except for bug fixes these properties and methods will not change their behavior in future versions of Xojo. Any methods not listed here are subject to change behavior or be removed at any time without notice.

close()

You should override this method to be notified when the parent view is being closed so your control can do any cleanup work it needs to do.

constructor(ID: string, events: string[])

You will call this method from your own control's Constructor method. Receives the control identifier and a list of events implemented at design-time.

controlName(): string

Returns the current control's control name. Only valid in debug mode.

DOMElement(suffix: string = ""): HTMLElement | null

Returns the DOM element whose id matches the controlID. the suffix will be added to the controlID before the lookup is performed. May be null if the element does not exist.

implementsEvent(eventName: string): boolean

Returns true if the passed event name was implemented at design-time or if the addHandler was used on the event. eventName must match the Xojo event name exactly.

panelIndex(): number

Returns the control's panel index if it is on a PagePanel or TabPanel.

parentControl(): XojoControl | null

Returns the control's parent control or view, or null if there isn't one.

parentLayout(): XojoPage | XojoDialog | null

Returns the Page or Dialog that the current control is on, or null if there isn't one.

scaleFactorChanged()

Called if the scale factor of the user's browser changes. Usually when the user drags the browser window from one monitor to another. The current value can be retrieved from window.devicePixelRatio. *Controls should override this method to be notified when this value changes.*

sendDeferredChanges()

This method will be called by the view if an action takes place where the user would have a reasonable expectation of all control values being up to date. For example, it would be called before the Default button's Pressed event is sent to the server. *You should override this method if your control doesn't send changes to the server immediately.*

triggerServerEvent(eventName: string, parameters: JSONItem | null = null, forced: boolean = false)

Sends an event to the server for processing. eventName *should* match the Xojo event name. parameters are of type JSONItem. The forced parameter should be used sparingly, it is meant for sending events that otherwise send important information to your control even if the event hasn't been explicitly implemented. For example, changes to the text in a textfield are always sent because the user has a reasonable expectation that the Value property of the field will be up to date, regardless of whether he or she implemented the ValueChanged event.

updateControl(data: string)

The updateControl method is called whenever the data returned from the Serialize event in your Xojo Code is sent to the browser. You **must** call super.updateControl(data) first to make sure the other control properties are updated properly.

XojoVisualControl

height(): number

Returns the IDE defined height of the control.

width(): number

Returns the IDE defined width of the control.

tabIndex: number (read-only property)

Returns the tab index for the control.

enabled(value: boolean) (setter)

Sets the enabled state of the control itself.

enabled: boolean (getter)

Returns the current enabled state of the control in terms of itself and all of its parent controls. if any parent is not enabled, this property will return false. If you need the enabled state of the control itself, you can get that from the mEnabled property.

interactiveElement(): HTMLElement | null

Returns the HTMLElement of the current control which is "interactive". By default, this looks inside the current control for a button, input or textarea tag, falling back to the element returned by DOMElement, but subclasses may override this method if DOMElement is incorrect.

focusElement(): HTMLElement | null

Returns the DOM element representing the part of the control that can get focus. By default this method returns the same element as interactiveElement.

render()

Called when it is time to render your control, usually due to a call to the Refresh method. When this method is first called, an empty div element has been created with the current control ID in its id attribute. It is important to remember that your control *may move* within the DOM depending on the placement of your control within other controls.

resize(width: number, height: number)

Sets the width and height of a control and then calls the resized method. *Controls may override if just setting the width and height are not accurate enough.*

resized()

Called when the control was resized. *Controls may override to be notified of size changes.*

parentResized()

Called when the parent control was resized. *Controls may override to be notified of parent control size changes.*

parentPanelShown()

Called when a control is on a tabpanel or pagepanel and the parent panel is shown. *Controls may override to be notified when the panel that the control is on is shown.*

setFocus()

sets the browser focus on the control's focusElement and updates the current tab engine state.

applyColorAndSize(classes: string[], colorPrefix: string)

adds the correct color for the indicator.

attachContextualMenu(parentID: string = "")

attaches the contextual menu to the current control. By default it connects the menu to the primary DOM element unless you pass a different value to parentID.

applyTooltip(el: HTMLElement)

Adds the necessary attributes to the specified element for a tooltip to appear

applyUserStyle(el:HTMLElement | null = null)

Applies the entries the user put into the control's Style property to the specified element (or the main div).

applyCSSStyle(propertyName: string, value: string, inherited: boolean, domEl: HTMLElement | null)

Called as each part of a user style is applied to the control. *Controls may override to fine-tune control over how individual styles are applied.*

Section 9

Specific Controls

In addition to writing completely custom controls, there are several built-in items that are designed to be augmented.

At the simplest level, **XojoProgressWheels** can have *any* SVG file applied to them. See Examples > Web > Controls > Custom ProgressWheels for some examples. The animated SVG files in that folder were created with Pixofield KeyShape.

WebListBoxCellRenderers are a way to make browser-side custom column types to offload some of the computing requirements. For instance, the WebListBoxDateTimeRenderer class allows the developer to just pass in a DateTime and the format to be used. The localization and formatting are both done on the browser using the moment.js JavaScript library. Please see XojoWebSDK.d.ts for more information about the methods you need to implement.

The JavaScript portion of the **WebFileUploader** has been specifically built so that the engine can be used independently from the UI so you can override the appearance without having to have different underlying functionality. Please see XojoWebSDK.d.ts for more information about the methods that are available and which need to be implemented.

Section 10

Javascript Globals

Methods

There are several global helper methods available to you for general use.

XojoWeb.DOMParentControl(el: HTMLElement): XojoControl | null

Walks up through the DOM looking for an element whose ID matches a ControlID in the session.

XojoWeb.getStyleForSelector(selector: string, attribute: string): string

Searches through the attached stylesheets in reverse order to find the specified selector and return the specified attribute.

XojoWeb.EncodeBase64(aString: string): string
XojoWeb.DecodeBase64(aString: string): string

Xojo-compatible Base64 encoding methods. In Xojo, be sure to set the length parameter to zero (0).

XojoWeb.getNamedControl(controlID: string): XojoControl | null

Returns either the XojoControl instance that matches the passed controlID or null if the control doesn't exist.

XojoWeb.sessionURL(): string

Returns the url for sending requests to the app.

XojoWeb.isDebugMode(): boolean

Returns True when the xojo app is running in Debug Mode

XojoWeb.XojoConsole.log(message: string, always: boolean = false)

A debug sensitive version of console.log which will only output the message when the Xojo app is running in debug mode or if the "always" parameter is set to true.

overrideMetaKey(keyname: string, target: EventTarget = document, callback: Function | null = null)

Method to intercept command/control-key combinations with an optional callback. Note, the available keys are determined by the browser so there will be certain things that you can't override, such as CMD/CTRL-Q.

Constants

XojoWeb.apiVersion

Version of the Xojo Web JavaScript framework. Currently version 7.

XojoWeb.xojoVersion

Version of the Xojo IDE that was used to build the currently running web app.

Section 11

Helper Classes

The Xojo Javascript Framework comes with a number of classes that you will use with XojoControl and XojoVisualControl. For instance, we made a javascript JSONItem class for use with the TriggerServerEvent method.

JSONItem

Methods

get(key: string): number | string | boolean | null | JSONItem | Array<string | number | boolean | null | JSONItem>
Gets a value using **key** as an index.

set(key: string, value: number | string | boolean | null | JSONItem | Array<string | number | boolean | null | JSONItem>)
Sets a value using **key** as an index.

lookup(key: string, defaultValue: number | string | boolean | null | JSONItem | Array<string | number | boolean | null | JSONItem>): number | string | boolean | null | JSONItem | Array<string | number | boolean | null | JSONItem>
Looks up a value by **key** with a specified fallback value if the key does not exist.

length(): number
Returns the number of items in the top level of the current JSONItem object.

serialize(): any
Returns the JSONItem as a string

Section 12

Important Notes

Xojo Events

It is important to remember that developers using your controls will not expect to see the Web SDK events in their controls. Any events that you do not implement should have at least a comment in the code so developers will not be confused (and possibly break your control). This includes:

- ExecuteEvent
- FrameworkPropertyChanged • SetupCSS
- SetupHTML
- SetupJavascriptFramework

Memory and Bandwidth

Memory and bandwidth usage is of the utmost importance for web applications because of the often limited resources of mobile devices, especially when combined with general Internet latency.

Memory

Web applications are all one page, so ensure your controls are not taking up valuable resources unnecessarily.

Bandwidth

If you are monitoring events that fire quickly (like mouse movement), you should throttle the events so that you're only sending as many as the user needs to accomplish the task. Overloading the framework with events not only makes your controls slow, it affects the responsiveness of the entire web page and application.

Make sure you are only sending the least amount of data needed. This is especially important for developers that are targeting mobile devices. For instance, if you are creating a toggle control, send "1" and "0" instead of "true" and "false". When dealing with mobile devices and those with limited Internet connections, every byte makes a difference.

3rd-Party Libraries

If you wrap a third-party control or library, be sure to document the usage license and provide links so your users can make an informed choice about their usage of the code.

CSS

It is important to remember that when a user positions a control in the IDE, they have a reasonable expectation that the control will appear in that position, using the dimensions that they specified. The only reason that you should be changing properties that would alter the position of your control is to make sure that the user's expectations will be maintained. For example, if you are wrapping a control that renders HTML above or below the supplied area, you should change the top and height properties so that the control will still render within the boundaries set by the user. Failure to follow these guidelines may cause your control to draw over (or under) other controls and make them unusable.

Xojo DOM & Framework

You may not add, remove or modify any part or property of the XojoWeb namespace or framework at design-time or runtime for any reason. You may not provide developer patches to the web framework or API.

Xojo reserves the right to modify the undocumented parts of the web framework without notice. Therefore you should not call undocumented parts of the web framework or rely on its internal structure. Doing so may cause your controls to fail in customer projects.

When it comes to the Document Object Model (DOM), you are only permitted to manipulate the content within your own controls. Changing the structure of built-in controls or the controls of other 3rd-Party developers is not permitted.

Shared Environment

Don't forget that your control will live in a shared environment, with the web framework itself and possibly also with other developer's controls. When writing code for your controls, be careful to make changes that only affect your controls. For instance, you should not be adding a class to the <body> tag or even to an entire WebPage. Styles should not be added to a particular control.

Section 13

Compatibility and Bugs

Testing and Compatibility

The standard framework controls have been designed to behave as consistently as possible on all supported browsers. For the best developer experience, your custom controls should do the same. In terms of browser compatibility, we are generally suggesting that users use the most recent version of the browsers available on their systems.

OS	Requirements
Windows	• Chrome (latest supported version) • Firefox (latest supported version) • Edge 12+
macOS	• Safari 9.0+ • Chrome (latest supported version) • Firefox (latest supported version)
Linux	• Chrome (latest supported version) • Firefox (latest supported version)
Raspberry Pi	• Raspbian Epiphany Web Browser • Chromium for Raspbian Pixel
iOS	• Mobile Safari (latest supported version)
Android	• Chrome for Android (latest supported version)

Bug Reports

For maximum efficiency in resolving framework bugs that affect your controls, your users should report bugs regarding your controls directly to you.

If you determine that there is an issue or problem in our web framework, **you** should file a bug report using Feedback and notify your users to mark it as a favorite and possibly add it to their top 5 cases.

Section 14

Tips & Tricks

Q: I'm getting errors when non-english users try to use text in my control because of non-ascii encodings. What can I do about this?

A: We specifically added Xojo-compatible methods to the JavaScript portion of the framework for encoding and decoding Base64 text. See XojoWeb.EncodeBase64 and XojoWeb.DecodeBase64. Don't forget to set the line length to zero when encoding text in your Xojo code.

Xojo Code:

```
Var encodedData as string = EncodeBase64(data, 0) // You must use 0 for the length parameter!
```

JavaScript code:

```
Var decodedData = XojoWeb.DecodeBase64(encodedData)
```

Section 15

TypeScript Quick Start

If you've never had experience with TypeScript, we thought we would give you a quick overview of how to install it and get it up and running on your machine.

The first thing to know about TypeScript is that the engine behind it is Node.js, which is an open source JavaScript runtime built on Chrome's V8 JavaScript engine. It's a highly versatile system with lots of projects based on it which if you haven't run into it yet, you probably run into it as a requirement soon.

Installing TypeScript

To install the prerequisites for TypeScript you'll need to first install Node.js.

1. Go to the Node.js download page and download the installer for your platform.

2. Install TypeScript. Installation instructions can be found on the TypeScript website here.

Your First Project

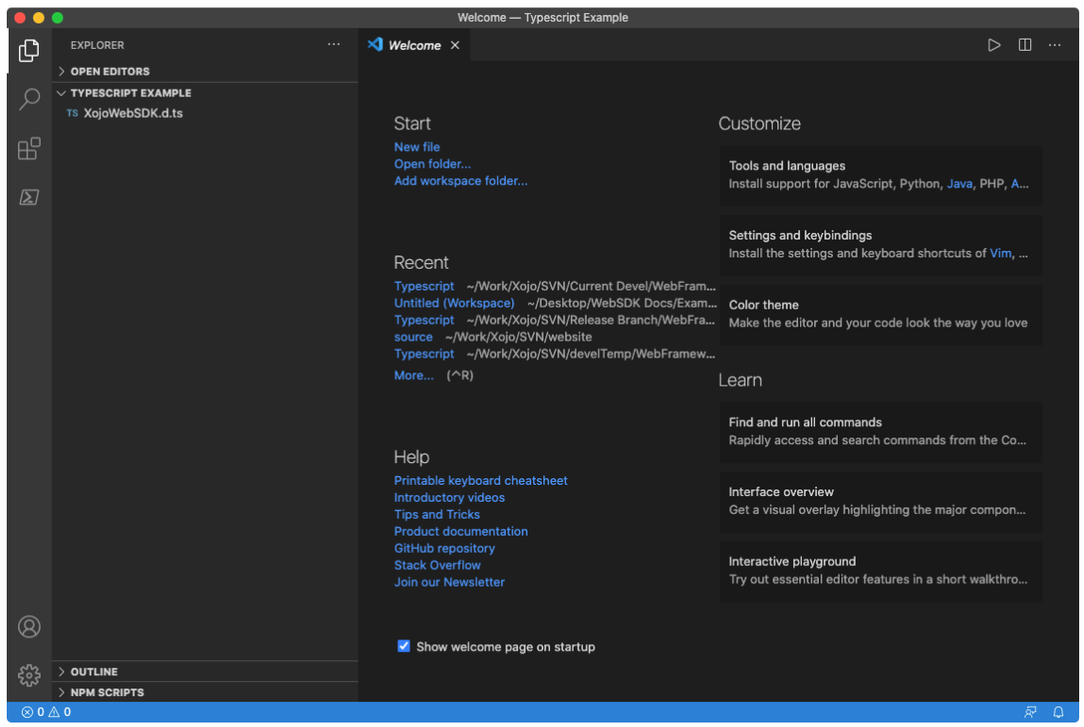
Getting TypeScript ready to develop Xojo WebSDK controls is fairly easy to do. You will only need to follow these instructions once per project.

1. Create a folder for your control. We like to create two folders within your custom control working folder called "typescript" and "javascript". The names are not important, but remembering and not confusing them is. We will refer to these folders as "your JavaScript folder" and "your TypeScript folder" respectively from now on.

   Note: It is important that these two folders not be nested or you will run into problems when compiling later.

2. Copy the XojoWebSDK.d.ts file into your TypeScript folder.

3. In TypeScript, select File > Open and choose your TypeScript folder. You should have a screen that looks like this:



4. To create the initial config file, you can use the Visual Studio Code Terminal. Open it by selecting View > Terminal.

5. Type the following line into the terminal:

   ```
   tsc --init
   ```

   This will create the initial configuration file.

6. Select the new tsconfig.json file in the file list and make some changes:

   6.1. Change the "target" option to "ES2015". We use this value to be able to target Edge 12 and above. You may also use ES2016 (Edge 13) or ES2017 (Edge 15) if your control needs it.

   6.2. We suggest uncommenting the "allowJs" and "checkJS" options. While not strictly required, it can catch errors in raw javascript files as well.

   6.3. Uncomment the "outFile" option. Set the value to the directory point to your JavaScript folder (from step 1). You may use an absolute or relative path here, so you could enter "../javascript" if the folder is right beside your TypeScript folder.

   6.4. We suggest uncommenting the "removeComments" option to keep your resulting compiled JavaScript as small as possible. Don't worry, they'll stay put in the TypeScript documents.

   6.5. We suggest uncommenting the "noImplicitReturns". Doing so makes it so that if you create a function with a return value but not all paths in that function return a value, it will be considered an error.

7. Now that you have the basic config set up, let's create a sample project file to see how compiling works.

   7.1. In the File explorer on the left, right-click and select New File.

   7.2. Use the name "mycontrol.ts". Note the **.ts** extension to indicate that this file is in TypeScript format.

7.3. Type the following code into the editor. This code declares your namespace within the Xojo Web Framework ecosystem. For more information about choosing a namespace and preventing conflicts with other developers, see the Namespaces section.

```
namespace Example {

}
```

7.4. Within the Example namespace you will create the class for your control. In this case we are creating a visual control.

```
export class myControl extends XojoWeb.XojoVisualControl {

}
```

The existence of the XojoWebSDK.d.ts file makes it possible for the editor to autocomplete the Xojo types.

7.5. To allow the initialization of instances of your control, you need to add a constructor within the class definition:

```
// Constructor for myControl
constructor(id: string, events: string[]) {
    super(id, events); // Call the super version first!
}
```

7.6. Save the file. The entire contents of the file should look like this:

```
namespace Example {
    export class myControl extends XojoWeb.XojoVisualControl {
        // Constructor for my controls
        constructor(id: string, events: string[]) {
            super(id, events);
        }
    }
}
```

Compiling

TypeScript has a nifty compiler mode called "watch" which will recompile your sourcecode any time you save a document. We highly recommend it because it lets you catch bugs as soon as they happen. To use it, go to the Terminal within Visual Studio Code and type:

```
tsc -w
```

At this point, the compiler will spin up and compile the example file we assembled above. In your JavaScript folder, you will now have a file called "mycontrol.js" which contains an ES2015-compatible version of your TypeScript code.

Testing

We suggest loading your JavaScript code from disk when testing so that you can quickly make changes in your TypeScript file, recompile and then re-run your project. You can accomplish this by adding a method which first loads from a constant in your control and if the user is running in debug mode and the constant is empty, load the code from disk. For final distribution, you only need to copy the contents of your JavaScript file into your javascript constant.